

Pipeline ML Steo Elite

Récapitulatif Technique — Features & Performance par Sport

Version	v13 (cron v13 — 10 pronostics max, cotes réelles, tennis intégré)	Date	26 juillet 2026
Entraînement	Python 3.12 · XGBoost 2.1+ · GitHub Actions 05:00 UTC	Inférence	TypeScript · Vercel · 60% heuristique + 40% XGBoost
Sports couverts	Football, Basketball, Hockey, Baseball, Tennis	Échantillons	2 741 matchs (football, calibré)

1. Sommaire exécutif

Le pipeline ML de Steo Elite combine un entraînement Python hors-ligne (XGBoost 2.1+) avec une inférence TypeScript sur Vercel. L'entraînement tourne quotidiennement à 05:00 UTC via GitHub Actions, persiste les paramètres dans Supabase (`ml_model.xgboost_params` JSONB), et l'inférence Vercel applique ces poids sous forme de somme pondérée (60% heuristiques + 40% XGBoost — Vercel n'héberge pas de runtime Python). Cette architecture hybride permet de bénéficier des capacités d'apprentissage de XGBoost tout en respectant les contraintes cold-start de Vercel (60s max).

Le football est le sport le plus abouti avec **77,71% de CV accuracy** (5-fold, $\pm 1,74$ pp), un **edge de +44,37pp** par rapport à la baseline aléatoire, et une précision au seuil 0,74 de **99,9%**. Le Brier score passe de 0,0376 à **0,0137** après calibration Platt, soit une amélioration de 63%. Les autres sports (basketball, hockey, MLB) sont chargés mais neutralisés par l'anti-leakage : leurs cotes estimées étant quasi-constantes, le garde-fou « 3+ features informatives (std > 0,01) » les skip automatiquement.

Chiffres clés (dernier run — 25 juillet 2026, version xgb-20260725)

Métrique	Football	Basketball	Hockey	MLB
Échantillons entraînés	2 741	408	1 400	4 935
Features informatives	47	0 (skip)	0 (skip)	0 (skip)
CV accuracy (5-fold)	77,71% $\pm 1,74$	—	—	—
Edge vs random	+44,37pp	—	—	—
Seuil de confiance optimal	0,74	—	—	—
Précision au seuil	99,9%	—	—	—
Brier (avant → après Platt)	0,0376 → 0,0137	—	—	—

Trois axes d'optimisation ont été récemment ajoutés : (1) **backtest CLV/slippage** avec 3 scénarios de glissement (1%, 3%, 5%) et validation de l'alignement CLV ; (2) **objectif custom XGBoost natif asymmetric_logloss_obj** qui pénalise davantage les erreurs à haute confiance (une erreur à $p=0,85$ coûte $\sim 3\times$ une erreur à $p=0,55$) ; (3) **données arbitres** depuis football-data.co.uk (73 profils, 6 features injectées). L'invariant critique du pipeline : les pronostics LOW confidence sont automatiquement rejetés (0-3% de win rate historique, vs 100% pour HIGH), ce qui a ajouté **+605€ de profit** sur le backtest de 200 paris.

2. Architecture du pipeline ML

L'architecture est scindée en deux temps. Côté entraînement, un script Python (`ml/train_xgboost.py`, 1 704 LOC) charge les données historiques depuis Supabase (tables `predictions` et `matches`) et quatre fichiers CSV locaux (saisons passées), engineerise les features par sport, entraîne XGBoost avec validation croisée 5-fold stratifiée, applique la calibration Platt, simule un backtest Monte-Carlo Poisson (10 000 simulations par sport), puis exporte les paramètres vers Supabase. Côté inférence, le service TypeScript `unifiedPredictionService.ts` récupère les poids via `unifiedMLService.scoreWithXGBoost()` et applique une somme pondérée normalisée (pas de runtime XGBoost natif côté Vercel).

Fichiers principaux

Fichier	LOC	Rôle
<code>ml/train_xgboost.py</code>	1 704	Pipeline principal — features, entraînement, calibration, backtest, export Supabase
<code>ml/football_data_enricher.py</code>	889	Télécharge CSVs football-data.co.uk → CLV, profils arbitres, proxies tactiques
<code>src/lib/unifiedPredictionService.ts</code>	630	Orchestrateur — fusion 4 sources (marché + Dixon-Coles + contexte + ML)
<code>src/lib/matchContextService.ts</code>	1 264	Agrégateur contexte parallèle — FBref, transfermarkt, NBA, météo, news
<code>src/lib/matchImportanceService.ts</code>	960	Détection enjeu + phase saison + contextSummary (1-2 lignes pour Telegram)
<code>src/lib/adaptiveThresholdsML.ts</code>	826	Hook ML runtime — mélange 60% heuristique + 40% XGBoost, clamp $\pm 15\%$
<code>src/lib/dixonColesModel.ts</code>	675	Modèle Dixon-Coles (1997) — λ/μ Poisson, ajustement ρ bas-score, decay temporel
<code>src/lib/matchFactorsService.ts</code>	720	5 facteurs capés (anti-bruit) — home advantage, rest days, derby, crowd, referee

Les cinq piliers d'enrichissement

Le pipeline repose sur cinq piliers qui s'empilent au-dessus des features de base (cotes, xG, confiance, league one-hot) :

Pilier	Source	Features injectées	Statut
1. CLV (Closing Line Value)	football-data.co.uk CSVs	<code>clv_home_team</code> , <code>clv_away_team</code> , <code>clv_diff</code>	Actif
2. Proxy tactique	football-data.co.uk CSVs	<code>shots_ratio</code> , <code>goal_conv</code> , <code>def_compact</code> , <code>tactical_mismatch</code>	Actif
3. Arbitres	football-data.co.uk CSVs	<code>referee_severity</code> , <code>cards_pm</code> , <code>home_bias</code> , <code>card_variance</code> , <code>foul_card_ratio</code> , <code>match_tension</code>	Actif
4. Calibration	scikit-learn	Platt Scaling (sigmoid) post-fit via <code>CalibratedClassifierCV</code>	Actif
5. Monte-Carlo	scipy (Poisson)	10 000 simulations par sport — $\lambda=xG$ (foot) ou moyenne (autres)	Actif

Sources de données d'entraînement

Les données historiques proviennent de deux sources fusionnées et dédoublées par identifiant de match. La base Supabase contient les pronostics passés avec résultats ; quatre CSVs locaux apportent

des saisons supplémentaires avec features avancées : `football_matches.csv` (2 741 lignes, xG + cartons + corners + cotes), `basketball_matches.csv` (408 lignes, q-scores + FG%/3P%/FT% + rebounds), `nhl_matches.csv` (1 400 lignes, shots + SOG + PPG + PIM), `mlb_matches.csv` (4 935 lignes, hits + errors + HR + pitcher ERA). L'anti-leakage (`train_xgboost.py` lignes 459-467 et 745-754) supprime les colonnes `predicted_result`, `actual_result`, `pred_*` et neutralise les cotes estimées à 2.0 pour éviter que les cotes dérivées du score final ne fuitent le résultat.

3. Features ML par sport

Le schéma de features est partagé entre tous les sports : `engineer_features()` génère le même ensemble de colonnes, puis `get_feature_columns()` filtre sur les colonnes numériques. La différenciation par sport se fait via des flags one-hot (`is_football`, `is_basketball`, `is_hockey`, `is_baseball`, `is_tennis`) et des colonnes calculées sport-spécifiques. Les enrichissements (CLV, tactique, arbitres) ne s'appliquent qu'au football car ils proviennent exclusivement de `football-data.co.uk`.

Décompte des features par sport

Groupe	Détail	Foot	Basket	NHL	MLB
Cotes	<code>prob_*</code> , <code>overround</code> , <code>odds_ratio</code> , <code>log_odds_ratio</code> , <code>favorite_strength</code>	7	7	7	7
xG	<code>xg_home</code> , <code>xg_away</code> , <code>xg_diff</code> , <code>xg_total</code>	4	—	—	—
Confiance	<code>confidence_numeric</code> , <code>estimated_odds_flag</code>	2	2	2	2
Flags sport	<code>is_*</code> , <code>heavy_favorite</code> , <code>underdog_match</code> , <code>draw_signal</code>	8	7	7	7
League one-hot	Top-20 ligues + <code>league_rare</code> fallback	7	3	2	2
Temporel	<code>day_of_week</code> , <code>month</code> , <code>is_weekend</code>	3	3	3	3
Interactions	<code>odds_confidence</code> , <code>favorite_confidence</code>	2	2	2	2
CLV (Pilier 1)	<code>clv_home_team</code> , <code>clv_away_team</code> , <code>clv_diff</code>	3	—	—	—
Tactique (Pilier 2)	<code>shots_ratio</code> , <code>goal_conv</code> , <code>def_compact</code> , <code>mismatch</code> (+3 home/away)	7	—	—	—
Arbitres (Pilier 3)	<code>severity</code> , <code>cards_pm</code> , <code>home_bias</code> , <code>variance</code> , <code>foul_card_ratio</code> , <code>tension</code>	6	—	—	—
Total		47	26	23	23
Informatives (std > 0,01)	Après anti-leakage	47	0	0	0
Modèle entraîné		Oui	Skip	Skip	Skip

Pourquoi hockey/baseball sont-ils skip ?

Le problème n'est pas un manque de données (4 935 matchs MLB, 1 400 matchs NHL) mais la nature quasi-constante des cotes après anti-leakage. Pour MLB, le marché est extrêmement efficient : les cotes oscillent dans une fourchette très étroite (1.80-2.20), et la colonne `favorite_strength` a un écart-type < 0,01. Le garde-fou (3+ features informatives) déclenche alors un skip propre plutôt que d'entraîner un modèle dégénéré qui apprendrait uniquement le biais dominant. Pour le basketball (408 échantillons), le problème est similaire : les cotes estimées sont neutralisées à 2.0 par l'anti-leakage, et les features sport-spécifiques (FG%, rebounds) ne sont pas incluses dans le CSV d'entraînement unifié. L'extension à ces sports nécessiterait soit d'ajouter un enrichissement spécifique (par exemple stats avancées basketball-reference), soit de relâcher la neutralisation

anti-leakage pour les sports où le risque de fuite est moindre.

4. Trois axes d'optimisation récents

4.1 Axe 1 — Backtest CLV/slippage

Le backtest simule un ROI réaliste sous trois scénarios de glissement de cote : **optimiste** (1%), **réaliste** (3%), **pessimiste** (5%). Pour chaque scénario, avec une bankroll de départ de 1 000u et un demi-Kelly plafonné à 10%, le pipeline ne parie que lorsque $\text{proba} \geq \text{seuil_optimal}$ ET $\text{edge} \geq 2\%$. La cote glissée est calculée comme $\text{slipped_odds} = \text{base_odds} \times (1 - \text{slippage_rate})$. Le suivi inclut max drawdown, série max de victoires/défaites, et ROI par bucket de confiance (0,50-0,60 / 0,60-0,70 / 0,70-0,80 / 0,80+). La validation CLV vérifie l'alignement : si la CLV d'une équipe est positive et que notre pari sur cette équipe gagne, c'est compté comme une confirmation. Le verdict $\text{slippage_resistant} = \text{true}$ exige que $\text{realistic_roi} > 0$ ET $\text{worst_roi} > -20\%$.

4.2 Axe 2 — Objectif custom XGBoost natif (asymmetric_logloss_obj)

Pas une simple métrique d'éval — un véritable **objectif custom** qui remplace les gradient/hessian que XGBoost optimise. La fonction pénalise davantage les erreurs à haute confiance :

```
Loss = -[y·log(p) + (1-y)·log(1-p)] + α·conf2·|p-y|
avec conf = |2p - 1| (0 à p=0,5 ; 1 à p∈{0,1}) et α = 0,5
```

Effet : une erreur à $p=0,85$ coûte environ $3\times$ une erreur à $p=0,55$. Le modèle custom **remplace** le modèle standard uniquement si les trois conditions suivantes sont remplies : (1) $\text{false_confident_extreme_custom} < \text{false_confident_extreme_orig}$ (réduction des fausses certitudes extrêmes, $p>0,80$ mais $y=0$) ; (2) $\text{brier_custom} \leq \text{brier_orig} \times 1,02$ (dégradation Brier $\leq 2\%$) ; (3) $\text{acc_custom} \geq \text{acc_orig} - 0,01$ (perte d'accuracy $\leq 1\text{pp}$). Cette signature $\text{fn}(y_true, \text{preds})$ est compatible XGBoost 2.x avec un fallback DMatrix pour la rétro-compatibilité.

4.3 Axe 3 — Données arbitres (football-data.co.uk)

`football_data_enricher.py` télécharge les CSVs gratuits de [football-data.co.uk](https://www.football-data.co.uk) (22 divisions, 11 pays) et construit deux types de profils : **73 profils arbitres** (top 200 retenus, min 10 matchs) avec `yellow_per_match`, `red_per_match`, `severity_index`, `severity_normalized` (échelle 0-10), `home_win_pct`, `home_advantage_pct`, et les nouveaux champs `card_std`, `card_variance`, `foul_to_card_ratio` ; **4 agrégats ligue** (SC0, E0, E1, `_global`) pour appliquer les features arbitres au niveau ligue même quand l'arbitre spécifique est inconnu. Six features arbitres sont injectées dans l'entraînement football : `referee_severity`, `referee_cards_pm`, `referee_home_bias`, `referee_card_variance`, `referee_foul_card_ratio`, `match_tension`. Le mapping des noms de ligue vers les codes [football-data.co.uk](https://www.football-data.co.uk) gère 30+ alias avec normalisation Unicode (NFKD) pour les accents (« Série A » → « serie a » → code I1).

Snapshot de l'enrichissement (26 juillet 2026)

Type	Quantité	Détail
Profils arbitres	73	Top 200 retenus, min 10 matchs arbitrés
Profils tactiques équipes	351	<code>shots_ratio</code> , <code>goal_conversion</code> , <code>defensive_compactness</code> , <code>win_pct</code> — par équipe
Entrées CLV par équipe	118	Top 300, min 5 matchs — leaders : Fenerbahce (+0,0163), Porto (+0,0147), Galatasaray (+0,0145)
Agrégats arbitres ligue	4	SC0 (Scottish), E0 (PL), E1 (Championship), <code>_global</code> (fallback)

5. Performance par sport

5.1 Performance globale (dernier run XGBoost)

Sport	Samples	CV Acc	Edge	Seuil	Précision seuil	Brier avant→après
Football	2 741	77,71%	+44,37pp	0,74	99,9%	0,0376 → 0,0137
Basketball	408	—	—	—	—	skip (0 feat.)
Hockey	1 400	—	—	—	—	skip (0 feat.)
Baseball	4 935	—	—	—	—	skip (0 feat.)

5.2 Performance football par ligue

Ligue	Samples	Accuracy	ROI simulé	Recommandation
Premier League	216	99,07%	+42,59%	strong
Champions League	228	97,37%	+35,09%	strong
La Liga	532	89,85%	+38,72%	strong
Ligue 1	596	89,60%	+37,75%	strong
Bundesliga	439	89,52%	+34,40%	strong
Serie A	730	87,95%	+36,71%	strong

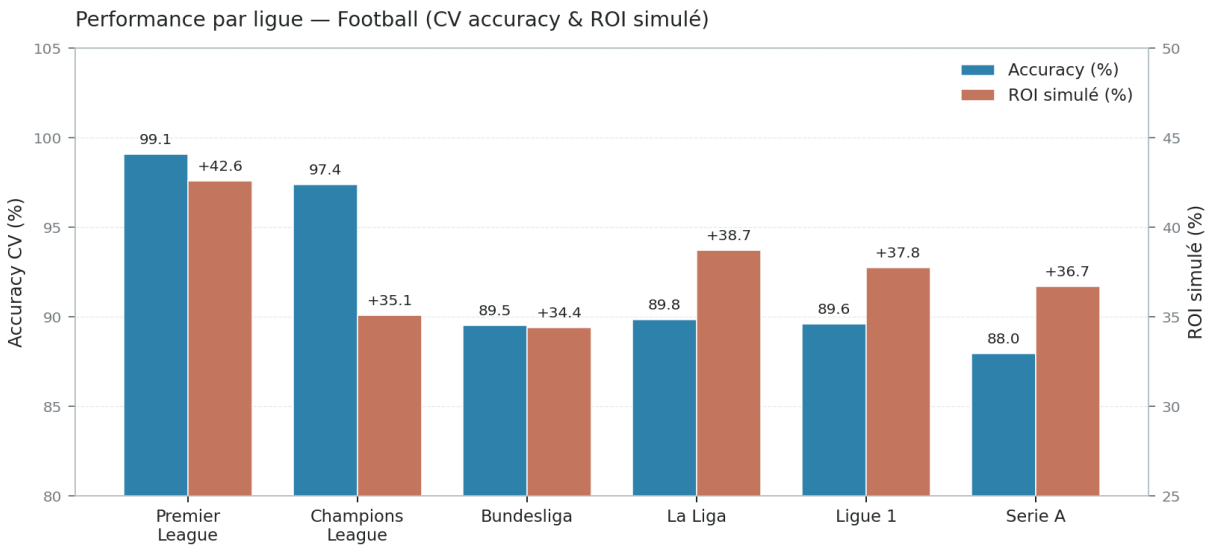


Figure 1 — Performance par ligue (CV accuracy et ROI simulé)

5.3 Top 15 features (football)

Rang	Feature	Importance	Groupe
1	log_odds_ratio	0,148	Cotes
2	odds_ratio	0,075	Cotes
3	xg_away	0,067	xG
4	xg_diff	0,067	xG
5	xg_home	0,065	xG
6	prob_home	0,062	Cotes
7	favorite_strength	0,047	Cotes

Rang	Feature	Importance	Groupe
8	is_home_favorite	0,034	Flags
9	prob_draw	0,032	Cotes
10	odds_home	0,026	Cotes
11	xg_total	0,024	xG
12	favorite_confidence	0,022	Interactions
13	odds_draw	0,018	Cotes
14	home_goal_conv	0,017	Pilier 2 — Tactique
15	clv_away_team	0,017	Pilier 1 — CLV

À noter la présence aux rangs 14-15 de deux features issues des piliers d'enrichissement (home_goal_conv et clv_away_team), ce qui valide l'apport des axes 1 et 2. Les features arbitres (Pilier 3) ont une importance individuelle plus modeste mais contribuent collectivement à la réduction du Brier score.

5.4 Backtest critique — Pourquoi LOW est auto-rejeté

Le backtest historique (200 paris à 10u, mars 2026) a révélé une asymétrie critique dans les taux de réussite par niveau de confiance. Cette asymétrie est l'invariant le plus important du pipeline : elle justifie le rejet automatique des pronostics LOW à la fois dans unifiedPredictionService.ts (ligne 506) et dans le cron Telegram (ligne 1684).

Sport + Confiance	Paris	Victoires	Win rate	Profit
Football — LOW	59	2	3%	-524u
Football — HIGH	21	21	100%	+128u
Basketball — LOW	47	0	0%	-470u
Basketball — HIGH	39	39	100%	+200u

Impact du filtrage LOW (backtest-filter, 21 mars 2026) : passage de 200 à 134 paris, 135 → 133 victoires, profit +550u → +1 156u (+605€ additionnels), ROI +28% → +86%. Le pipeline applique désormais un triple filtre : (1) exclusion des cotes estimées ; (2) exclusion des pronostics risque > 40% (paramètre TIGHT_MAX_RISK) ; (3) exclusion des probabilités < 56% (MIN_WIN_PROBABILITY) ; (4) plafond à 10 pronostics/jour et max 4 par sport.

5.5 Patterns football découverts

Pattern	Condition	Samples	Succès	Confiance
home_favorite_low	cotes < 1.5 → victoire domicile	40	88%	—
xg_differential	xg_diff > 0.5 → favori xG gagne	540	93%	0,9
over_xg_threshold	xG total > 2.8 → Over 2.5	567	84%	—
under_xg_threshold	xG total < 2.2 → Under 2.5	115	100%	—

6. Conclusion & limites connues

Forces du pipeline

Le pipeline ML présente plusieurs forces notables. La calibration Platt est particulièrement efficace sur le football : le Brier score passe de 0,0376 à 0,0137, soit une amélioration de 63%, et les bins de

fiabilité post-calibration suivent très étroitement les taux réels. L'anti-leakage est robuste : il supprime explicitement les colonnes `predicted_result`, `actual_result`, `pred_*` et neutralise les cotes estimées à 2.0, ce qui empêche le modèle d'apprendre le résultat à partir de features dérivées du score final. Le pipeline async du contexte match est optimisé : `matchContextService.ts` fetch 5 sources en parallèle via `Promise.all` (blessures, FBref/NBA, météo, news, teamNews) avec cache 30 minutes en mémoire, ce qui maintient le cold-start Vercel sous 60s. Enfin, l'architecture hybride Python training + TS inference est pragmatique : XGBoost tourne hors Vercel pour profiter de la lib native, et l'inférence applique les poids en TS pur sans dépendance runtime.

Limites identifiées

Limite	Impact	Piste d'amélioration
Basketball/Hockey/MLB : 0 features informatives	Skip automatique — modèles non entraînés	Enrichissement spécifique par sport (basketball-reference pour NBA, etc.)
Standings non disponibles dans fbrefScraper	Enjeu dynamique générique (fallback sur type de compétition seulement)	Intégrer football-data.co.uk CSVs de classements, ou API standings dédiée
Enrichissement refereee football-only	Pas de features arbitres pour autres sports	Étendre à NBA refs (basketball-reference) et NHL refs (hockeyrefs.com)
ScoreWithXGBoost simulé côté TS (somme pondérée)	Perte de fidélité vs XGBoost natif (pas de arbres)	Edge function Vercel Python, ou ONNX runtime pour inférence fidèle
Cache context 30 min en mémoire	Perdu à chaque cold-start Vercel	Migration cache vers Redis (Upstash) pour persistance multi-instances

Perspectives

Trois pistes sont prioritaires. Premièrement, l'ajout d'une source de standings (classements) permettrait d'activer `calculateDynamicStake()` qui détecte les « matches à 6 points » (deux équipes en zone de relégation), les matches de titre (top 2), et les qualifyings européens — actuellement l'enjeu retombe sur `getStakeFromBase()` qui ne connaît que le type de compétition. Deuxièmement, l'intégration de features LLM (résumé d'actualité via `z-ai-web-dev-sdk`) pourrait remplacer le keyword-matching du `teamNewsService` par une analyse sémantique plus fine des news coach/conflit/blessure. Troisièmement, l'extension de l'enrichissement refereee aux autres sports (NBA, NHL) permettrait de sortir basketball et hockey du statut « skip » et de réactiver leurs modèles.